

The SuperClaude Field Guide

A friendly, complete handbook for the free SuperClaude add-on to Claude Code: what it does, how to set it up, and exactly which commands to type for almost any situation.

1. In your terminal

```
pipx install superclaude
```

Installs the framework.

2. Still in the terminal

```
superclaude install
```

Adds 30 commands to Claude Code.

3. In Claude Code

```
/sc:recommend "your goal"
```

Get told exactly what to run.

What is inside the framework, and inside this guide

30

 **Commands**

Slash commands that start structured jobs

19

 **Specialists**

Expert roles Claude can take on

7

 **Modes**

Working styles that adapt to the task

10

 **Helper tools**

Optional add-ons for docs, browsers, search

59

 **Recipes**

Ready-made command sequences

Edition 4.0, October 2026 **Covers** SuperClaude v4.3.0 **Verified** against source, 3 October 2026

Level Complete beginner to expert

SuperClaude is an independent, MIT-licensed community project. It is not made, endorsed or supported by Anthropic. This guide describes third-party software.

Contents

Every entry is a link in the PDF and on screen. Page numbers refer to the printed page.

	How to use this guide	3
	Part I: Get started	
01	What SuperClaude is	4
02	Install and set up	5
03	Your first session	8
	Part II: The toolkit	
04	The 30 commands	10
05	The safety ladder	17
06	The specialists (agents)	18
07	The seven modes	21
08	Helper tools (MCP servers)	22
09	Flags: the fine-tuning dials	24
10	The house rules	26
	Part III: Recipes	
11	Find the right recipe	28
12	Recipes: starting and planning	30
13	Recipes: building features	32
14	Recipes: fixing problems	36
15	Recipes: improving existing code	40
16	Recipes: understanding and documenting	43
17	Recipes: shipping and working across days	45
18	Recipes: research, business and specifications	48
19	Recipes: special set-ups	49
	Part IV: Help and reference	
20	Troubleshooting	51
21	Frequently asked questions	52
22	Glossary	53
23	Pocket reference cards	55
24	How this guide was verified	57

② BEFORE YOU START

How to use this guide

You do not need to read it all. Find your starting point below.

This guide is written for people who have never typed a SuperClaude command. Every technical word is explained the first time it appears, and again in the glossary. Experienced users can jump straight to the reference chapters and recipes.

🔗 Choose your path



Brand new

Read chapters 1 to 3 in order. Then pick a recipe from chapter 11. See page 4.



Using it daily

Keep chapter 4 (commands) and chapter 23 (pocket cards) open. See page 10.



Going deeper

Chapters 7 to 10 explain modes, tools, flags and rules. See page 21.



Not a programmer

Research and business recipes (chapter 18) need no coding at all. See page 48.

🔖 Labels you will see

Two kinds of label appear throughout. The square ones tell you what a command does to your files. The round ones tell you how experienced you need to be. Every label carries a word as well as a colour, so the guide reads correctly in black and white.

👁 Look only

Reads and reports. Never changes a file.

📄 Plans only

Produces a document or plan, then stops for you.

✍ Changes files

Edits your project. Review the result before committing.

📖 Memory

Saves or restores session context.

● Beginner

Safe to try on day one.

● Intermediate

Comfortable after a week or two.

● Expert

For large or critical work; uses more tokens.

🔧 How commands are shown

Grey boxes hold things you type. Each is labelled with where to type it: the **terminal** (the text window where you install software) or **Claude Code** (the chat window where you use SuperClaude, with commands that start with `/`). Text after `#` is an explanation you do not type. Words in quotes are your own description; parts in `[square brackets]` are optional.

```
/sc:analyze src --focus security # look for security problems in the src folder
```

CLAUDE CODE

PART I

Get started

What SuperClaude is, how to install it, and a guided first session.



01

What SuperClaude is

4



02

Install and set up

5



03

Your first session

8

CHAPTER 01

What SuperClaude is

A plain-language picture before any commands.

Claude Code is Anthropic's tool that lets Claude work directly on the files in a project folder: reading code, writing code and running commands. **SuperClaude** is a free add-on, made by an independent community, that gives Claude Code a set of ready-made working methods.

Without SuperClaude, you explain every time how you want Claude to approach a job. With it, you type a short command such as `/sc:brainstorm` and Claude follows a tried-and-tested procedure: it asks the right questions, involves the right expertise, uses the right tools and stops at sensible checkpoints so you stay in control.



Under the hood it is just text

SuperClaude is a collection of instruction files written in Markdown (a simple text format). Installing it copies those files into a hidden folder called `~/.claude/` in your home directory. It does not change which Claude model you use, your plan or your bill.

The five building blocks

Think of the framework as a stack. You touch the top layer; everything below supports it.



Commands

The verbs. You type `/sc:something` to start a structured job such as planning, building or testing.



Specialists

The experts. Each command calls in the right “hat”: security engineer, technical writer, architect.



Modes

The working style. Questioning when ideas are vague, compressed when space is tight, step-by-step for big jobs.



Helper tools

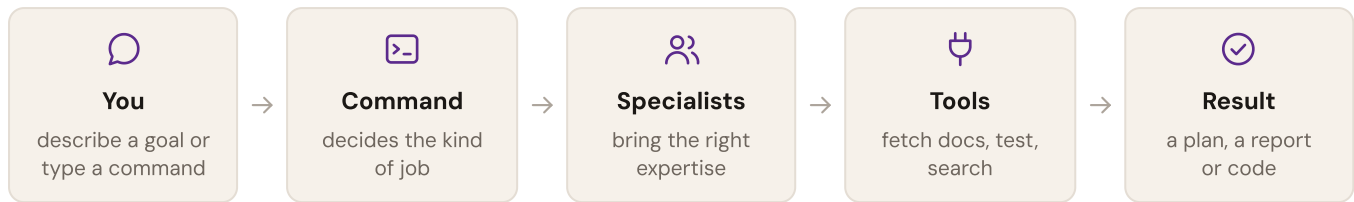
The instruments. Optional add-ons that read official docs, drive a browser, search the web or remember sessions.



House rules and flags

The standing policy and the fine-tuning dials that govern everything above.

S What happens when you type a command



💡 A simple way to picture it

Imagine hiring a small consulting firm for each job. The **command** is the type of engagement: a design review, a build, a bug hunt. The **specialists** are the consultants who turn up. The **helper tools** are their reference books and instruments. The **house rules** are the firm's policies: check your work, never fake a passing test, never leave a job half done.

🛡️ The single most important idea

Many commands **stop on purpose**. A planning command produces a plan and waits; it will not quietly start changing your code. That is a safety feature. Chapter 5, [the safety ladder](#) (page 17), shows exactly which commands look, which plan and which change files.

✅ What SuperClaude is not

- It is not an Anthropic product, and Anthropic does not support it.
- It is not a separate app. It only works inside Claude Code.
- It is not magic. It is good working methods, written down. You still review what Claude produces.

↓ CHAPTER 02

Install and set up

From nothing to a working installation, then optional extras.

Installation takes about five minutes. The core needs only Python; the helper tools are optional and can be added later, one at a time.

☰ What you need first

You need	Required?	Why	Check it
Claude Code, installed and signed in	Yes	SuperClaude adds to it; it cannot run alone	<code>claude --version</code>
Python 3.10 or newer	Yes	The installer is a Python package	<code>python3 --version</code>
pipx (recommended) or pip	Yes	Installs the package cleanly	<code>pipx --version</code>
Node.js 18 or newer	For most helper tools	Seven of the helper tools run on Node	<code>node --version</code>
uv	Only for Serena	Runs the Serena memory tool	<code>uv --version</code>
Docker	Only for the AIRIS gateway	Hosts Airis Agent and MindBase	<code>docker --version</code>

**Older guides say Python 3.8**

That is out of date. Version 4.3.0 requires Python 3.10 or newer, and the installer will refuse anything older.

Install in four steps**1 Install the package (terminal)**

```

pipx install superclaude      # recommended
# or, if you don't have pipx:
pip install superclaude

```

TERMINAL

2 Copy the commands and specialists into Claude Code (terminal)

```

superclaude install          # 30 commands + 20 specialist files

```

TERMINAL

Commands go to `~/.claude/commands/sc/`, specialists to `~/.claude/agents/`.

3 Check the installation (terminal)

```

superclaude install --list    # shows every command and specialist
superclaude doctor            # health check
superclaude version           # should print 4.3.0

```

TERMINAL

4 Restart Claude Code completely, then test it (Claude Code)

```

/sc:help

```

CLAUDE CODE

If a table of commands appears, you are ready. If not, see page 51.

Adding helper tools (optional)

Helper tools make some commands much better: official documentation lookup, real browser testing, web research and memory between sessions. Every command still works without them. The `superclaude mcp` command installs them for you.

```

superclaude mcp --list          # see what is available and installed
superclaude mcp                 # choose interactively
superclaude mcp --servers context7 --servers sequential-thinking
superclaude mcp --servers tavily --dry-run    # preview without installing
superclaude mcp --scope project              # this project only (default: user)

```

TERMINAL

Three tools need a free or paid key from their provider. The installer asks for it, or you can set it in your environment first:

Tool	Key name	Get it from
Tavily (web search)	TAVILY_API_KEY	app.tavily.com
Magic (UI components)	TWENTY_FIRST_API_KEY	21st.dev
Morphllm (bulk edits)	MORPH_API_KEY	morphllm.com

**Where to start**

Begin with **Context7** and **Sequential**: no keys, big improvement. Add **Serena** when you want memory between sessions, and **Tavily** when you start using `/sc:research`.

The AIRIS gateway (advanced)

The installer also offers the AIRIS MCP Gateway, which bundles more than 60 tools, including Airis Agent and MindBase, behind one connection. It needs Docker running on your machine.

```
superclaude mcp --servers airis-mcp-gateway
```

TERMINAL

Updating

```
pipx upgrade superclaude      # or: pip install --upgrade superclaude
superclaude update            # overwrite the old command files with the new ones
```

TERMINAL

**Running install again does not update**

`superclaude install` skips any file that already exists, so after an upgrade it silently keeps the old commands. Use `superclaude update` (the same as `superclaude install --force`). Then restart Claude Code.

Optional: the confidence-check skill

The `/sc:agent` controller can use a small “confidence check” skill that scores how sure Claude is before it starts building.

```
superclaude install-skill confidence-check
```

TERMINAL

Windows notes

- Use pip or pipx in PowerShell. The `install.sh` script is for macOS and Linux (or Git Bash).
- If `superclaude` is “not recognized”, run `python -m superclaude install` instead, or add Python's Scripts folder to your PATH.
- With several Python versions, pick one explicitly: `py -3.12 -m pip install superclaude`.

Uninstalling

```
pipx uninstall superclaude      # or: pip uninstall superclaude
rm -rf ~/.claude/commands/sc    # removes the 30 commands
claude mcp remove <tool-name>   # for each helper tool you added
```

TERMINAL

**Remove agents one by one**

The specialists live in `~/.claude/agents/` alongside any agents of your own. Delete only the SuperClaude files (their names are listed on page 18, plus `deep-research.md`) rather than the whole folder.

 CHAPTER 03

Your first session

A guided twenty-minute walk-through you can follow along.

This walk-through takes about twenty minutes and is completely safe: nothing changes your files until step 5, and even then you approve every change. Open Claude Code in any project folder, ideally one tracked by git.

1 Open your project and load context

```
/sc:load
```

CLAUDE CODE

Claude reads the project. On a first run there is nothing saved yet, which is fine.

2 Not sure what to do? Ask

```
/sc:recommend "I want to understand this project and then fix a small bug" --  
expertise beginner
```

CLAUDE CODE

You get a suggested sequence of commands with an explanation of each. Notice it changes nothing.

3 Get a tour of the code

```
/sc:analyze . --focus architecture --depth quick  
/sc:explain src --level basic
```

CLAUDE CODE

The first gives a structured overview; the second explains it in everyday language.

4 Diagnose something, without fixing it

```
/sc:troubleshoot "describe a small problem you know about" --type bug --trace
```

CLAUDE CODE

You receive a likely cause and ranked options. Still nothing has changed.

5 Apply a fix, with your approval

```
/sc:troubleshoot "the same description" --type bug --fix  
/sc:test
```

CLAUDE CODE

Claude asks before applying anything risky, then runs your tests.

6 Commit and save

```
/sc:git commit --smart-commit  
/sc:save --checkpoint
```

CLAUDE CODE

A tidy commit message is written for you, and the session is saved so tomorrow's `/sc:load` picks up here.

**You can always just talk**

Commands are shortcuts, not a requirement. Type `/sc:pm` followed by a goal in plain English, for example `/sc:pm "add a dark mode toggle to the settings page"`, and the project-manager specialist works out which steps and experts are needed.

**Nothing runs on its own**

Some SuperClaude files describe `/sc:pm` and `/sc:agent` as starting automatically. That applies only to the future plugin edition. With today's pip or pipx install, nothing happens until you type a command.

 A routine worth keeping

When	Type	Why
Start of a session	<code>/sc:load</code>	Restore yesterday's context
Unsure what to do	<code>/sc:recommend "..."</code>	Get a suggested path
Every 30 minutes, and before risky steps	<code>/sc:save --checkpoint</code>	A restore point
Before calling it done	<code>/sc:reflect --type completion</code>	Catch anything missed
End of a session	<code>/sc:save --type all</code>	Keep everything for next time

PART II

The toolkit

Every command, specialist, mode, tool and flag, explained in plain language.

 04	The 30 commands	10	 05	The safety ladder	17
 06	The specialists (agents)	18	 07	The seven modes	21
 08	Helper tools (MCP servers)	22	 09	Flags: the fine-tuning dials	24
 10	The house rules	26			

 CHAPTER 04

The 30 commands

What each slash command does, when to use it and what comes next.

There are 30 commands. You will use about eight of them most days. They are grouped below by what you are trying to do, and every card shows the same five things: what it does, a label for its effect on your files, the full syntax, a realistic example and the usual next step.

 Short and long names

Some older pages write `/implement` instead of `/sc:implement`. They are the same command. Because the files install into a folder called `sc`, Claude Code shows them with the `sc:` prefix. Always type the `/sc:` form.

 Getting oriented

Commands that help you find your way, pick the right tool or run a whole task for you.

`/sc:recommend`

 Look only Beginner

Describe your problem in everyday words and it suggests which commands to run, in what order.

Syntax `/sc:recommend "[what you want]" [--estimate] [--alternatives] [--stream] [--community] [--expertise beginner|intermediate|expert]`

Example `/sc:recommend "my website is slow, help me speed it up" --expertise beginner`

Then Run the sequence it suggests.

 Understands requests in several languages, not just English.

`/sc:help`

 Look only Beginner

Prints a table of commands and flags. It never changes anything.

Syntax `/sc:help`

Example `/sc:help`

Then Pick a command from the list.

 Its table is hand-written and lists 24 of the 30 commands. It omits `/sc:agent`, `/sc:index-repo`, `/sc:pm`, `/sc:recommend`, `/sc:research` and `/sc`. For the full list, use `superclaude install --list` in your terminal.

/sc

👁 Look only

● Beginner

A short menu of the framework. Typing `/sc` on its own shows what is available.

Syntax `/sc` or `/sc:[command] [arguments]`

Example `/sc`

Then Choose a command.

/sc:pm

✎ Changes files

● Beginner

The project manager. Give it a goal in plain English; it breaks the work down and calls the right specialists and tools for you.

Syntax `/sc:pm [request] [--strategy brainstorm|direct|wave] [--verbose]`

Example `/sc:pm "add a password-reset email to the login page"`

Then Review what it produced, then `/sc:test`.

🕒 Its file describes it as the always-on default layer. In a normal pip/pipx install nothing runs by itself: type `/sc:pm` (or `/sc:agent`) to start it.

/sc:agent

✎ Changes files

● Intermediate

A session controller. It checks your git status, then investigates, plans, builds and reviews a task, and refuses to start building until it is at least 90% confident.

Syntax `/sc:agent` then describe your task

Example `/sc:agent` then: `"fix the broken date filter on the reports page"`

Then Let it finish its self-review, then commit.

🕒 Its file says it starts automatically at session start; that only applies to the plugin edition (planned for v5). With the pip install, type it yourself.

/sc:task

✎ Changes files

● Intermediate

Runs one clearly defined job from start to finish, coordinating specialists, then stops.

Syntax `/sc:task [action] [target] [--strategy systematic|agile|enterprise] [--parallel] [--delegate]`

Example `/sc:task refactor "authentication module" --strategy systematic --delegate`

Then `/sc:test`, then `/sc:git`.

/sc:spawn

📄 Plans only

● Expert

Splits a very large job into epics, stories and tasks, and decides what can run in parallel. It plans; it does not execute.

Syntax `/sc:spawn [complex-task] [--strategy sequential|parallel|adaptive] [--depth normal|deep]`

Example `/sc:spawn "migrate the monolith to services" --strategy adaptive --depth deep`

Then `/sc:task` or `/sc:implement` for each piece.

/sc:select-tool

👁 Look only

● Expert

Decides which editing tool suits a change: Serena for meaning-aware edits (like renames), Morphllm for fast find-and-replace across files.

Syntax `/sc:select-tool [operation] [--analyze] [--explain]`

Example `/sc:select-tool "rename getUser to fetchUser everywhere" --explain`

Then Run the edit with the suggested flag (--serena or --morph).

📌 Scores above 0.6 go to Serena, below 0.4 to Morphllm, otherwise to Claude's built-in tools.

📄 Planning and ideas

Turn a fuzzy idea into a clear plan. None of these write code.

/sc:brainstorm

📄 Plans only

● Beginner

Asks you questions, one round at a time, until a vague idea becomes a clear list of requirements.

Syntax `/sc:brainstorm [idea] [--strategy systematic|agile|enterprise] [--depth shallow|normal|deep] [--parallel]`

Example `/sc:brainstorm "an app that reminds people to water their plants" --depth normal`

Then `/sc:design` or `/sc:workflow`.

📌 Stops before any design or code.

/sc:estimate

👁 Look only

● Beginner

Estimates time, effort or complexity, with a breakdown and a confidence level.

Syntax `/sc:estimate [target] [--type time|effort|complexity] [--unit hours|days|weeks] [--breakdown]`

Example `/sc:estimate "user login with Google" --type time --unit days --breakdown`

Then `/sc:workflow`.

📌 Estimates are guidance, not promises.

/sc:workflow

📄 Plans only

● Intermediate

Turns a feature description or requirements document into a step-by-step build plan with dependencies.

Syntax `/sc:workflow [requirements-file | feature] [--strategy systematic|agile|enterprise] [--depth shallow|normal|deep] [--parallel]`

Example `/sc:workflow docs/requirements.md --strategy agile --depth normal`

Then `/sc:implement`.

📌 Saves the plan to `claudedocs/workflow_*.md`.

<> Designing and building

Design the shape of something, then build, package and tidy it.

/sc:design

[Plans only](#)[● Intermediate](#)

Designs the structure of something: the architecture, an API, a component or a database. Produces diagrams or specifications, not working code.

Syntax **/sc:design** [target] [--type architecture|api|component|database] [--format diagram|spec|code]

Example **/sc:design** orders-api --type api --format spec

Then /sc:implement.

/sc:implement

[Changes files](#)[● Beginner](#)

Writes the code for a feature. This is the main “build it” command.

Syntax **/sc:implement** [feature] [--type component|api|service|feature] [--framework react|vue|express] [--safe] [--with-tests]

Example **/sc:implement** "contact form with email validation" --type component --framework react --with-tests

Then /sc:test.

ⓘ Does not deploy or commit.

/sc:build

[Changes files](#)[● Beginner](#)

Compiles and packages your project using the build setup it already has.

Syntax **/sc:build** [target] [--type dev|prod|test] [--clean] [--optimize] [--verbose]

Example **/sc:build** --type prod --clean --optimize

Then /sc:test or deploy.

ⓘ Will not invent new build scripts or install missing tools.

/sc:improve

[Changes files](#)[● Intermediate](#)

Makes existing code better: cleaner, faster or easier to maintain. Small safe fixes happen automatically; bigger changes need your approval.

Syntax **/sc:improve** [target] [--type quality|performance|maintainability|style] [--safe] [--interactive]

Example **/sc:improve** src/checkout --type performance --interactive

Then /sc:test.

ⓘ Formatting and unused imports are fixed automatically. Logic, public APIs or multi-file changes wait for your go-ahead.

/sc:cleanup

[✎ Changes files](#)[● Beginner](#)

Removes dead code and unused imports, and tidies file structure.

Syntax `/sc:cleanup [target] [--type code|imports|files|all] [--safe|--aggressive] [--interactive]`

Example `/sc:cleanup src --type imports --safe`

Then `/sc:test`.

ⓘ Anything still referenced indirectly, exported publicly or used in config is left for you to approve.

/sc:git

[✎ Changes files](#)[● Beginner](#)

Runs git for you and writes tidy, conventional commit messages.

Syntax `/sc:git [operation] [args] [--smart-commit] [--interactive]`

Example `/sc:git commit --smart-commit`

Then Push or open a pull request.

ⓘ Asks before anything destructive.

🔍 Checking and fixing

Look for problems, run tests, find out why something broke.

/sc:analyze

[👁 Look only](#)[● Beginner](#)

Reads your code and reports problems by severity: quality, security, performance or architecture. It never edits.

Syntax `/sc:analyze [target] [--focus quality|security|performance|architecture] [--depth quick|deep] [--format text|json|report]`

Example `/sc:analyze src --focus security --depth deep`

Then `/sc:improve` or `/sc:cleanup`.

/sc:test

[👁 Look only](#)[● Beginner](#)

Finds your test runner, runs your existing tests and reports coverage.

Syntax `/sc:test [target] [--type unit|integration|e2e|all] [--coverage] [--watch] [--fix]`

Example `/sc:test --type all --coverage`

Then `/sc:troubleshoot` for failures, `/sc:git` when green.

ⓘ It does not write new tests. To add tests, use `/sc:implement ... --with-tests`. With `--fix` it may suggest repairs, and asks before touching test files.

/sc:troubleshoot

[👁 Look only](#)[● Beginner](#)

Diagnoses why something is broken and lists ranked fixes. Without `--fix` it changes nothing.

Syntax `/sc:troubleshoot [issue] [--type bug|build|performance|deployment] [--trace] [--fix]`

Example `/sc:troubleshoot "login button does nothing on Safari" --type bug --trace`

Then Run again with `--fix` to apply the chosen fix.

ⓘ Read-only by default. With `--fix` it still asks for confirmation for anything risky, then verifies.

/sc:reflect

👁 Look only

● Intermediate

A sanity check: is the current approach still on target, and is the task really finished?

Syntax `/sc:reflect [--type task|session|completion] [--analyze] [--validate]`

Example `/sc:reflect --type completion`

Then Fix any gaps, then `/sc:save`.

📌 Works best with the Serena tool installed.

📖 Explaining and documenting

Understand code and write it down for others.

/sc:explain

👁 Look only

● Beginner

Explains code or a concept at the level you choose, from beginner to advanced.

Syntax `/sc:explain [target] [--level basic|intermediate|advanced] [--format text|examples|interactive] [--context domain]`

Example `/sc:explain src/auth/session.ts --level basic --format examples`

Then Ask follow-up questions.

/sc:document

✎ Changes files

● Beginner

Writes documentation for one specific thing: code comments, an API reference or a user guide.

Syntax `/sc:document [target] [--type inline|external|api|guide] [--style brief|detailed]`

Example `/sc:document src/api --type api --style detailed`

Then Review and commit.

/sc:index

✎ Changes files

● Intermediate

Generates documentation for the whole project: a docs set, an API index, a structure map or a README.

Syntax `/sc:index [target] [--type docs|api|structure|readme] [--format md|json|yaml]`

Example `/sc:index . --type readme --format md`

Then Review and commit.

📌 Keeps your hand-written docs rather than overwriting them.

/sc:index-repo

✎ Changes files

● Intermediate

Writes a short map of your repository (PROJECT_INDEX.md and .json) so future sessions read the map instead of every file.

Syntax `/sc:index-repo [mode=update | mode=quick]`

Example `/sc:index-repo mode=update`

Then `/sc:load`.

📌 The file documents itself as `/index-repo`, but it installs into the `sc` folder, so the working name is `/sc:index-repo`. Claimed saving: about 58K tokens down to 3K per session (project's own figure).

Remembering between sessions

Save and restore your progress so tomorrow starts where today ended.

/sc:load

 Memory

 Beginner

Opens your project and restores what was saved last time. Run it at the start of a work session.

Syntax `/sc:load [target] [--type project|config|deps|checkpoint] [--refresh] [--analyze]`

Example `/sc:load`

Then Describe today's task.

 Needs the Serena tool to restore memory. Without it, it still reads the project.

/sc:save

 Memory

 Beginner

Saves what was learned and done, so the next `/sc:load` can pick up seamlessly.

Syntax `/sc:save [--type session|learnings|context|all] [--summarize] [--checkpoint]`

Example `/sc:save --type all --checkpoint`

Then Close the session.

 Needs the Serena tool.

Expert panels and research

Get a simulated panel of experts, or a cited research report.

/sc:research

 Plans only

 Beginner

Searches the web and writes a cited research report. Depth controls how many rounds of follow-up searching it does.

Syntax `/sc:research "[question]" [--depth quick|standard|deep|exhaustive] [--strategy planning|intent|unified]`

Example `/sc:research "best way to add payments to a small online shop in 2026" --depth standard`

Then `/sc:design` or `/sc:brainstorm`.

 Saves to claudedocs/research_*.md. Uses the Tavily search tool when installed.

/sc:business-panel

 Plans only

 Intermediate

Runs a document past nine simulated business thinkers (Porter, Christensen, Drucker, Godin, Kim & Mauborgne, Collins, Taleb, Meadows, Doumont).

Syntax `/sc:business-panel [document | text] [--experts "a,b,c"] [--mode discussion|debate|socratic|adaptive] [--focus domain] [--synthesis-only] [--structured] [--verbose]`

Example `/sc:business-panel @business_plan.md --mode debate --experts "porter,taleb,collins"`

Then `/sc:design` or `/sc:workflow`.

 Simulated perspectives, not real advice from those people.

/sc:spec-panel

Plans only

Intermediate

A panel of simulated software-engineering authors scores a specification for clarity, completeness, testability and consistency (0–10 each).

Syntax `/sc:spec-panel [spec | @file] [--mode discussion|critique|socratic] [--experts "a,b"] [--focus requirements|architecture|testing|compliance] [--iterations N] [--format standard|structured|detailed]`

Example `/sc:spec-panel @api_spec.yml --mode critique --focus requirements`

Then Revise the spec, then `/sc:design`.

❗ Experts include Wiegers, Adzic, Cockburn, Fowler, Nygard, Newman, Hohpe, Crispin, Gregory and Hightower.

✓ CHAPTER 05

The safety ladder

Which commands only look, which plan, and which change your files.

Every command sits on one rung of this ladder. The higher the rung on the page, the less it can touch. If you are nervous, stay on the first two rungs: they cannot change a single file.

👁 Look only

Reads and reports. Never changes a file.

`/sc:recommend` `/sc:help` `/sc` `/sc:select-tool` `/sc:estimate`
`/sc:analyze` `/sc:test` `/sc:troubleshoot` `/sc:reflect`
`/sc:explain`

📄 Plans only

Produces a document or plan, then stops for you.

`/sc:spawn` `/sc:brainstorm` `/sc:workflow` `/sc:design`
`/sc:research` `/sc:business-panel` `/sc:spec-panel`

💾 Memory

Saves or restores session context.

`/sc:load` `/sc:save`

✎ Changes files

Edits your project. Review the result before committing.

`/sc:pm` `/sc:agent` `/sc:task` `/sc:implement` `/sc:build`
`/sc:improve` `/sc:cleanup` `/sc:git` `/sc:document` `/sc:index`
`/sc:index-repo`

💡 Even “Changes files” commands ask first

`/sc:improve` and `/sc:cleanup` only apply trivial fixes on their own (formatting, unused imports). Anything bigger waits for your approval. Add `--interactive` to approve every step, or `--safe` for the most conservative behaviour.

🔗 Where each command stops, and what to run next

Commands are designed as a relay: each one hands over at a defined point so you can review before the next leg begins.

Command	Produces	Stops before	Run next
<code>/sc:brainstorm</code>	Requirements list	Any design or code	<code>/sc:design</code> , <code>/sc:workflow</code>
<code>/sc:design</code>	Architecture, API or schema spec	Writing code	<code>/sc:implement</code>
<code>/sc:workflow</code>	Step-by-step plan	Running anything	<code>/sc:implement</code>
<code>/sc:estimate</code>	Time or effort estimate	Starting work	<code>/sc:workflow</code>
<code>/sc:analyze</code>	Findings report	Editing code	<code>/sc:improve</code> , <code>/sc:cleanup</code>
<code>/sc:troubleshoot</code>	Diagnosis and options	Fixing (unless <code>--fix</code>)	<code>/sc:troubleshoot --fix</code>
<code>/sc:research</code>	Cited report	Acting on findings	<code>/sc:design</code> , <code>/sc:brainstorm</code>
<code>/sc:business-panel</code>	Expert analysis	Implementing advice	<code>/sc:design</code> , <code>/sc:workflow</code>
<code>/sc:spec-panel</code>	Scored spec review	Rewriting the spec for you	<code>/sc:design</code>
<code>/sc:spawn</code>	Task hierarchy	Executing the tasks	<code>/sc:task</code> , <code>/sc:implement</code>
<code>/sc:implement</code>	Working code	Committing or deploying	<code>/sc:test</code> , <code>/sc:git</code>
<code>/sc:task</code>	One finished task	Starting another unasked	<code>/sc:test</code> , <code>/sc:git</code>

🔗 CHAPTER 06

The specialists (agents)

Nineteen expert “hats” Claude can wear, and how to call one.

A specialist (SuperClaude calls them **agents**) is a set of instructions that makes Claude think like a particular expert. Most of the time they switch on automatically, based on your words and the command you use. You can also call one by name.

CLAUDE CODE

```
@agent-security-engineer "review the login flow for weaknesses"
/sc:implement "user profile API" --agent backend # ask a command to favour one
```

Each specialist also has deliberate limits. A front-end architect will not design your database; it hands that to the back-end architect. If a specialist declines part of a job, that is why.

 **Twenty files, nineteen roles**

Installation copies 20 files. Two of them, `deep-research-agent` and `deep-research`, are the same role: the second is a short 31-line summary of the first. Either name works.

<> Builders

They design and construct systems.

<> backend-architect

Designs reliable servers, APIs and databases. Thinks about failure, security and data integrity first.

Good for: API design, database schemas, authentication, caching

Hands off: Front-end UI, infrastructure

<> frontend-architect

Builds interfaces people can actually use: accessible, fast and responsive on every screen.

Good for: Components, accessibility (WCAG 2.1 AA), Core Web Vitals, design tokens

Hands off: Back-end APIs, databases

<> system-architect

Plans the big picture so the system still works at ten times today's size.

Good for: Component boundaries, scaling, patterns, technology choices

Hands off: Detailed implementation code

<> devops-architect

Automates building, deploying and monitoring, and assumes things will fail.

Good for: CI/CD, infrastructure as code, containers, observability

Hands off: Business logic, product decisions

<> python-expert

Writes production-quality Python from the first line: tested, secure, idiomatic.

Good for: Test-driven development, pyproject tooling, profiling

Hands off: Quick-and-dirty shortcuts

✓ Quality guardians

They test, secure, speed up and simplify.

✓ security-engineer

Thinks like an attacker to find weaknesses before someone else does.

Good for: OWASP Top 10, threat modelling, auth, encryption, compliance

Hands off: Trading security for convenience

✓ quality-engineer

Looks past the happy path for the edge cases that break things.

Good for: Test strategy, edge cases, automation, coverage

Hands off: Writing business logic

✓ performance-engineer

Measures first, then optimises. Never guesses where the slowness is.

Good for: Load times, API latency, memory and CPU, benchmarks

Hands off: Optimising without measurement

✓ refactoring-expert

Simplifies code in small, safe steps without changing what it does.

Good for: Reducing complexity, removing debt, SOLID patterns

Hands off: Adding features mid-refactor

✓ self-review

A second pair of eyes after building: were tests really run, edge cases covered, requirements met, rollback needed?

Good for: Post-build checklist reports

Hands off: Long narratives

🔍 Investigators

They dig into causes, requirements and the wider world.

🔍 root-cause-analyst

Follows evidence, tests competing explanations, and documents how the cause was found.

Good for: Logs, error patterns, hypotheses, fix plans

Hands off: Jumping to conclusions

🔍 requirements-analyst

Asks “why” before “how” until a vague wish becomes a testable specification.

Good for: Discovery, user stories, acceptance criteria

Hands off: Choosing technology

🔍 deep-research-agent

Researcher and investigative journalist: plans searches, follows leads up to five hops, cites sources, scores confidence.

Good for: Powers `/sc:research`

Hands off: Paywalls, private data, unsupported speculation

🗨 Teachers and writers

They explain and document.

🗨 technical-writer

Writes for the reader, not the author. Clear beats complete.

Good for: API docs, user guides, troubleshooting pages

Hands off: Implementing features

🗨 learning-guide

Teaches understanding step by step, linking new ideas to what you already know.

Good for: Tutorials, worked examples, comprehension checks

Hands off: Handing over answers without explanation

🗨 socratic-mentor

Guides you to discover principles yourself through questions, drawing on Clean Code and classic design patterns.

Good for: Code-review coaching, pattern recognition

Hands off: Giving the answer first

🧩 Specialists and coordinators

They manage sessions, maps and expert panels.

🧩 pm-agent

The project manager that keeps a session on track: restores context, plans in Plan–Do–Check–Act cycles, records successes and mistakes.

Good for: Writes patterns to docs/patterns/ and mistake reports to docs/mistakes/

Hands off: Retrying a failed approach without finding out why

🧩 repo-index

Compresses your repository into a short map so sessions stay cheap. Refreshes it when older than seven days.

Good for: PROJECT_INDEX.md and PROJECT_INDEX.json

Hands off: Editing code

🧩 business-panel-experts

Supplies the nine simulated business thinkers used by `/sc:business-panel`.

Good for: Discussion, debate and Socratic modes

Hands off: Real-world advice from the named people

Panels of named experts

Two commands simulate panels of well-known authors. `/sc:business-panel` uses nine business thinkers; `/sc:spec-panel` uses software-engineering authors. Both choose suitable experts automatically, or you can name them with `--experts`. They are simulations of published ideas, not the real people. For specifications, the focus you choose picks the panel: **requirements** brings Wiegers, Adzic and Cockburn; **architecture** Fowler, Newman, Hohpe and Nygard; **testing** Crispin, Gregory and Adzic; **compliance** Wiegers, Nygard and Hightower.

CHAPTER 07

The seven modes

Working styles that switch on by themselves, or when you ask.

Modes are working styles rather than jobs. They decide *how* Claude behaves while a command runs: asking questions, showing its reasoning, compressing its answers. They switch on by themselves when the situation calls for it, and you can force one with a flag.

Brainstorming

Switch on: `--brainstorm`

Kicks in when: Vague requests: “maybe”, “thinking about”, “not sure”.

What changes: Asks questions instead of assuming, then writes a requirements brief.

Introspection

Switch on: `--introspect`

Kicks in when: Error recovery, “why did you do that?”, tricky decisions.

What changes: Shows its reasoning step by step with small markers, so you can see how it decided.

Deep Research

Switch on: `/sc:research`

Kicks in when: Research questions and anything past the model’s knowledge cutoff.

What changes: Plans an investigation, searches in parallel, cites sources and states confidence.

Task Management

Switch on: `--task-manage`

Kicks in when: Jobs with more than three steps, or touching more than two folders or three files.

What changes: Breaks work into phases, tracks progress, saves checkpoints.

Orchestration

Switch on: `--orchestrate`

Kicks in when: Jobs that need several tools, or could run in parallel.

What changes: Picks the best tool for each step and runs independent steps together.

Token Efficiency

Switch on: `--uc` / `--token-efficient`

Kicks in when: Context more than 75% full, or very large jobs.

What changes: Answers in a compressed, symbol-heavy style that uses 30–50% fewer tokens.

Business Panel

Switch on: `/sc:business-panel`

Kicks in when: Strategy documents and business plans.

What changes: Simulates a panel of business thinkers discussing, debating or questioning.

If replies suddenly turn terse and full of symbols

That is Token Efficiency mode, which starts automatically when the conversation is over 75% full. It is expected. Start a fresh Claude Code session to reset, or ask Claude to answer normally.



Check your own installation

The mode, rule and flag definitions ship inside the SuperClaude package, but the 4.3.0 installer copies only the command and specialist files into `~/ .claude/`. Much of the mode behaviour is repeated inside individual commands, so it works in practice. If a mode flag seems to be ignored, describe the behaviour in words as well.

CHAPTER 08

Helper tools (MCP servers)

Ten optional add-ons: what each one is for and how to install it.

MCP stands for Model Context Protocol, the standard way of plugging extra tools into Claude Code. SuperClaude knows how and when to use ten of them. None is required: every command falls back to Claude's built-in abilities when a tool is missing.

Context7

Looks up official, version-specific documentation for libraries and frameworks.

Use it for: Using React, Next.js, Django and the like correctly for the version you have.

Needs: Node.js 18+

Flag: `--c7`

```
superclaude mcp --servers context7
```

Sequential

A step-by-step reasoning engine for problems with many moving parts.

Use it for: Complex debugging, architecture decisions, anything with `--think`.

Needs: Node.js 18+

Flag: `--seq`

```
superclaude mcp --servers sequential-thinking
```

Magic

Generates modern UI components from the 21st.dev pattern library.

Use it for: Buttons, forms, modals, cards, navigation bars.

Needs: Node.js 18+ and a 21st.dev API key (TWENTY_FIRST_API_KEY)

Flag: `--magic`

```
superclaude mcp --servers magic
```

Playwright

Drives a real browser: clicks, fills forms, takes screenshots, runs end-to-end tests.

Use it for: Testing login flows, checking a page renders, accessibility checks.

Needs: Node.js 18+

Flag: `--play`

```
superclaude mcp --servers playwright
```

Chrome DevTools

Profiles a live page: load speed, layout shift, console errors, network.

Use it for: "Why is this page slow?" and "what is that console error?"

Needs: Node.js 18+

Flag: `--chrome`

```
superclaude mcp --servers chrome-devtools
```

Serena

Understands code by meaning (symbols, references) and stores project memory between sessions.

Use it for: Renaming across files, `/sc:load` and `/sc:save`, large codebases.

Needs: uv (Python tool runner)

Flag: `--serena`

```
superclaude mcp --servers serena
```

Morphllm

Applies the same pattern edit across many files, quickly and cheaply.

Use it for: Style rules, swapping one call for another everywhere.

Needs: Node.js 18+ and a Morph API key (MORPH_API_KEY)

Flag: `--morph`

```
superclaude mcp --servers morphllm-fast-apply
```

Tavily

Web search for current information.

Use it for: `/sc:research`, news, prices, anything newer than the model's training.

Needs: Node.js 18+ and a Tavily API key (TAVILY_API_KEY)

Flag: `--tavily`

```
superclaude mcp --servers tavily
```

Airis Agent

Confidence checks, research and repository indexing via the AIRIS MCP Gateway.

Use it for: Checking a decision before building it.

Needs: Docker (installed with the AIRIS gateway)

Flag: `-`

```
superclaude mcp --servers airis-mcp-gateway
```

MindBase

Long-term semantic memory across conversations (PostgreSQL + pgvector).

Use it for: Searching past sessions by meaning.

Needs: Docker (installed with the AIRIS gateway)

Flag: `-`

```
superclaude mcp --servers airis-mcp-gateway
```

Why some lists say eight

The installer offers eight tools individually. Airis Agent and MindBase come bundled in the AIRIS gateway, which is why the project's own README counts eight while ten are documented.

Choosing between the two editing tools

If the change is...	Use	Because
Renaming or moving a function, class or symbol	Serena <code>--serena</code>	It understands meaning and finds every real reference
The same text pattern in many files	Morphllm <code>--morph</code>	It is fast and cheap for bulk find-and-replace
Not sure	<code>/sc:select-tool "..." - -explain</code>	It scores the job and picks for you

Tools that work well together

Pairing	Who does what
Context7 then Sequential	Context7 fetches the official docs; Sequential works out the plan.
Context7 then Magic	Context7 supplies framework rules; Magic builds a compliant component.
Sequential then Playwright	Sequential designs the test plan; Playwright clicks through it.
Playwright then Chrome DevTools	Playwright drives the user journey; DevTools measures its speed.
Serena then Morphllm	Serena works out what must change; Morphllm applies the bulk edit.
Tavily then Sequential	Tavily gathers sources; Sequential turns them into an analysis.
Tavily then Playwright	Tavily finds pages; Playwright reads the JavaScript-heavy ones.
Tavily then Serena	Tavily does the research; Serena remembers it for next time.

🔍 How research rates its sources

Tier 1 · 0.9–1.0 Academic, government, official docs	Tier 2 · 0.7–0.9 Established industry publications	Tier 3 · 0.5–0.7 Reputable blogs, community wikis	Tier 4 · 0.3–0.5 Forums and social media
--	--	---	--

🔧 CHAPTER 09

Flags: the fine-tuning dials

Small switches you add to any command to change how it works.

Flags are small switches added to the end of a command, always starting with two dashes. They let you say “think harder”, “use the browser”, “be careful” or “keep it short” without writing a paragraph. Most also switch on automatically when they fit.

```
/sc:analyze src --think-hard --c7          # deep analysis, with official docs
/sc:build --safe-mode --validate --focus security
```

CLAUDE CODE

🌀 Thinking depth at a glance



🔧 Behaviour modes

Change how Claude approaches the work.

Flag	What it does	Switches on by itself when
--brainstorm	Ask questions first, assume nothing.	Vague requests
--introspect	Show the reasoning behind each decision.	Error recovery, learning
--task-manage	Break into phases and track progress.	More than 3 steps
--orchestrate	Choose tools carefully, run independent steps in parallel.	Multi-tool jobs
--token-efficient	Compressed replies, 30–50% fewer tokens.	Context over 75% full

🔧 Tool switches

Turn specific helper tools on or off.

Flag	What it does	Switches on by itself when
--c7 / --context7	Official documentation lookup.	Library or framework questions
--seq / --sequential	Structured multi-step reasoning.	Complex debugging, design
--magic	UI component generation.	Buttons, forms, layouts

Flag	What it does	Switches on by itself when
<code>--morph / --morphllm</code>	Fast pattern edits across files.	Bulk changes
<code>--serena</code>	Meaning-aware code tools and memory.	Renames, big codebases
<code>--play / --playwright</code>	Real-browser testing.	End-to-end and visual checks
<code>--chrome / --devtools</code>	Live page profiling.	Slow pages, console errors
<code>--tavily</code>	Web search.	Current information
<code>--frontend-verify</code>	Playwright + DevTools + Serena together.	UI debugging
<code>--all-mcp</code>	Every tool at once.	Very complex, multi-domain jobs
<code>--no-mcp</code>	No external tools; built-ins and web search only.	Simple tasks, saving tokens

Thinking depth

How hard Claude thinks before acting. Deeper costs more tokens.

Flag	What it does	Switches on by itself when
<code>--think</code>	Standard structured analysis, about 4K tokens. Adds Sequential.	A few connected parts
<code>--think-hard</code>	Deep analysis, about 10K tokens. Adds Sequential and Context7.	System-wide questions
<code>--ultrathink</code>	Maximum depth, about 32K tokens. Adds every tool.	Redesigns, critical bugs

Execution control

How the work is carried out.

Flag	What it does	Switches on by itself when
<code>--delegate [auto files folders]</code>	Split work across parallel sub-agents.	More than 7 folders or 50 files
<code>--concurrency [1-15]</code>	Cap how many things run at once.	Limited resources
<code>--loop</code>	Repeat improve-and-check cycles.	“Polish”, “refine”
<code>--iterations [1-10]</code>	Set the number of cycles.	With <code>--loop</code>
<code>--validate</code>	Assess risk before acting.	Risky or production work
<code>--safe-mode</code>	Most cautious execution; also turns on <code>--uc</code> .	Production, critical systems

Output and focus

What the answer covers and how it looks.

Flag	What it does	Switches on by itself when
<code>--uc / --ultracompressed</code>	Symbol-based, very short replies.	Tight context
<code>--scope [file module project system]</code>	How wide to look.	Limiting or widening analysis

Flag	What it does	Switches on by itself when
<code>--focus</code> <code>[performance security quality architecture accessibility testing]</code>	One area of concern.	Targeted reviews



Three flags are not in `/sc:help`

`--chrome`, `--tavily` and `--frontend-verify` are defined in the framework's flag file but missing from the table `/sc:help` prints. Flags are instructions Claude reads, so they usually work; if one seems ignored, also say it in words ("use Chrome DevTools").

🚩 When flags disagree

Rule	What wins
Safety first	<code>--safe-mode</code> beats <code>--validate</code> , which beats any speed or optimisation flag.
You win	A flag you type always beats automatic detection.
Deepest wins	<code>--ultrathink</code> beats <code>--think-hard</code> , which beats <code>--think</code> .
Off means off	<code>--no-mcp</code> overrides every individual tool flag.
Widest scope wins	system beats project, which beats module, which beats file.

📖 CHAPTER 10

The house rules

Standing rules Claude follows whichever command you use.

Whichever command is running, Claude follows standing rules. Their spirit: **evidence over assumptions, working code over documentation, efficiency over wordiness**. They explain why it checks git first and refuses to switch off a failing test.

⚠️ Never compromised

Rule	In practice
Git safety	Check <code>git status</code> first, work on a feature branch rather than main, commit before risky steps.
Find the real cause	Never skip, disable or comment out a failing test to get a green result.
Plan parallel work up front	Decide early which steps can run side by side and which must wait.
Know today's date	Check the actual date before making date-based claims.
Respect the project	Check which libraries are already used before adding one; follow existing conventions.

Strong preferences

Rule	In practice
Finish what you start	No placeholder code, no “TODO: implement” left in core features.
Build only what was asked	No surprise login systems, pipelines or dashboards added “just in case”.
Clean up after yourself	Delete temporary scripts and files before finishing.
Professional honesty	No hype like “blazingly fast”, no invented numbers, honest trade-offs.
Tidy file placement	Reports in <code>claudedocs/</code> , tests in <code>tests/</code> , scripts in <code>scripts/</code> .
Good habits (when practical)	Consistent naming for the language; the most capable tool available, such as multi-file edits over one-by-one edits.

Before changing a file or starting a feature

Editing?

Read the file first; follow its patterns.

Creating?

Check the folder structure; put it where it belongs.

Scope unclear?

Brainstorm first:
`/sc:brainstorm`.

Tests exist?

Run them first:
`/sc:test`.

PART III

Recipes

Fifty-nine copy-ready command sequences for real situations.

 11	Find the right recipe	28	 12	Recipes: starting and planning	30
 13	Recipes: building features	32	 14	Recipes: fixing problems	36
 15	Recipes: improving existing code	40	 16	Recipes: understanding and documenting	43
 17	Recipes: shipping and working across days	45	 18	Recipes: research, business and specifications	48
 19	Recipes: special set-ups	49			

CHAPTER 11

Find the right recipe

Start from your situation and jump straight to the steps.

Recipes are tested sequences of commands for real situations. Find the sentence below that sounds like you, then jump to the recipe.

Which kind of command do I need?

I only have an idea

`/sc:brainstorm` then `/sc:workflow`

I know what to build

`/sc:design` then `/sc:implement`

Something is broken

`/sc:troubleshoot`, add `--fix` when sure

It works but could be better

`/sc:analyze` then `/sc:improve`

I don't understand it

`/sc:explain` or `/sc:index-repo`

I have no idea

`/sc:recommend "describe it"`

I want to...

Your situation	Start with	Recipes
I have an idea but no plan	First day on a brand-new project	P1 p.30 P2 p.30
I need to know if an idea is worth it	"Is this even worth building?" check	P3 p.31
I can't decide between two technologies	Choose between technologies	P4 p.31
I need to build a feature	A full-stack feature, end to end	B1 p.32 B2 p.33
I need a piece of user interface	A React component, generated and checked	B3 p.33 B8 p.35
I need to change the database	Design a database and add a migration	B5 p.34
I need tests for existing code	Add tests to code that has none	B9 p.35

Your situation	Start with	Recipes
Something is broken	Quick bug fix	F1 p.36 F11 p.39
Production is down	Urgent production problem	F2 p.36
The build fails	The build is broken	F3 p.37
Tests fail randomly	Failing or flaky tests	F4 p.37
The site or API is slow	A slow web page	F6 p.38 F7 p.38
There are errors in the browser console	Console errors or a broken layout	F9 p.39
An upgrade broke my app	An upgrade broke things	F10 p.39
The code is messy	Refactor safely	I1 p.40 I2 p.40
I'm worried about security	Security audit	I3 p.41
I need it to be accessible	Accessibility review	I4 p.41
I need to rename or replace something everywhere	Rename something everywhere	I7 p.42 I6 p.42
I've just joined a project	Get to know an unfamiliar codebase	L1 p.43
I don't understand this code	Understand one confusing file	L2 p.43 L3 p.44
I need documentation	Write a README	L4 p.44 L5 p.44
I want feedback before a code review	Prepare for a code review	L7 p.45
I'm ready to commit or release	Commit and push	S4 p.47 S5 p.47
My work spans several days	Daily start and finish routine	S1 p.45 S2 p.46
I need research with sources	A cited research report	R1 p.48 R2 p.48
I want my business plan challenged	Stress-test a business plan	R3 p.48
I'm running out of tokens	Working on a small token budget	X1 p.49
I want nothing changed without my say-so	Maximum caution: change nothing without asking	X3 p.50

How to read a recipe

Each recipe has an ID (such as [F1](#)), a level, a line saying when to use it, numbered steps, the result you can expect, and sometimes a tip. Run the steps in order. The grey line under each step tells you what it produces, so you know what to look at before moving on.



Adapt freely

Change the quoted descriptions and folder names to match your project. Folder names like `src/checkout` are examples. If a recipe uses a helper tool you have not installed, the step still runs, just with less power.

CHAPTER 12

Recipes: starting and planning

From a blank page to a plan you trust. 6 recipes.

P1 First day on a brand-new project

● Beginner

When: You have an idea and an empty folder.

- 1 `/sc:load`
opens the project folder (nothing to restore yet)
- 2 `/sc:brainstorm "a booking app for a small yoga studio" --depth normal`
answers your questions → requirements list
- 3 `/sc:design booking-app --type architecture --format diagram`
overall shape of the system
- 4 `/sc:workflow "booking app MVP" --strategy agile`
step-by-step build plan
- 5 `/sc:save --checkpoint`
remembers all of the above for tomorrow

✓ **Result:** A requirements list, an architecture diagram and a build plan, with nothing written to your code yet.

💡 Answer brainstorm questions in plain sentences. "I don't know yet" is a perfectly good answer.

P2 Turn a vague idea into a solid specification

● Beginner

When: You can describe what you want in a sentence but not in detail.

- 1 `/sc:brainstorm "let customers track their parcel" --depth deep`
deep question rounds → requirements
- 2 `/sc:spec-panel @claudedocs/requirements.md --mode critique --focus requirements`
experts score it and list gaps
- 3 `/sc:spec-panel @claudedocs/requirements.md --mode discussion --iterations 2`
second pass after you revise
- 4 `/sc:estimate "parcel tracking" --type effort --breakdown`
how big the job is

✓ **Result:** A reviewed specification with a scored quality report and an effort estimate.

💡 Use the file name brainstorm actually printed; the path above is an example.

P3 “Is this even worth building?” check

● Intermediate

When: Before investing weeks in an idea.

1 `/sc:research "demand and competitors for plant-care reminder apps" --depth standard`
cited market report

2 `/sc:business-panel @claude/docs/research_plant_apps.md --mode debate --experts "christensen,porter,taleb"`
experts argue for and against

3 `/sc:estimate "plant-care app MVP" --type complexity`
how hard the first version is

✓ **Result:** Evidence, a stress-tested argument and a complexity estimate, so you can decide go or no-go.

P4 Choose between technologies

● Intermediate

When: You are torn between two frameworks, databases or hosting options.

1 `/sc:recommend "real-time chat for 5,000 users" --alternatives`
candidate stacks in a comparison table

2 `/sc:research "Supabase vs Firebase for real-time chat 2026" --depth deep`
evidence for the shortlist

3 `/sc:design chat-system --type architecture --format spec --think-hard`
architecture using the winner

✓ **Result:** A reasoned technology decision backed by sources, plus a first architecture.

P5 Scope a minimum viable product (MVP)

● Beginner

When: You want the smallest version that is still useful.

1 `/sc:brainstorm "invoice generator for freelancers" --strategy agile --depth shallow`
core needs only

2 `/sc:workflow "invoice generator MVP" --strategy agile --depth shallow`
lean build plan

3 `/sc:estimate "invoice generator MVP" --unit days --breakdown`
days per piece

✓ **Result:** A short, realistic MVP plan with a day-by-day estimate.

💡 The house rules forbid building extras you did not ask for, so scope stays small.

P6 Plan a large, multi-part initiative

● Expert

When: A big job spanning several areas or teams.

1 `/sc:spawn "move billing from the monolith to its own service" --strategy adaptive --depth deep`

epic → story → task breakdown

2 `/sc:workflow claudedocs/billing_epic.md --strategy enterprise --parallel`

dependency-mapped plan

3 `/sc:task execute "billing service: phase 1" --strategy enterprise --delegate`

builds phase 1 only

4 `/sc:reflect --type task --validate`

checks phase 1 against the plan

✓ **Result:** A full hierarchy of work and the first phase delivered, checked against the plan.

</> CHAPTER 13

Recipes: building features

*Writing new code, from a single component to a full feature. 10 recipes.***B1 A full-stack feature, end to end**

● Intermediate

When: A feature that needs both a back end and a screen.

1 `/sc:brainstorm "team invitations by email" --depth normal`

requirements

2 `/sc:design invitations --type api --format spec`

API contract

3 `/sc:design invite-dialog --type component --format spec`

UI component spec

4 `/sc:workflow "team invitations" --strategy systematic`

build order

5 `/sc:implement "invitations API" --type api --framework express --safe --with-tests`

back end + tests

6 `/sc:implement "invite dialog" --type component --framework react --with-tests`

front end + tests

7 `/sc:test --type all --coverage`

everything runs green?

8 `/sc:git commit --smart-commit`

tidy commit

✓ **Result:** A working, tested feature on both sides, committed with a clear message.

B2 Build an API

● Intermediate

When: You need REST or GraphQL endpoints.1 `/sc:design orders-api --type api --format spec`

endpoints, inputs, errors

2 `/sc:implement "orders API" --type api --framework express --safe --with-tests --c7`

code that follows official framework patterns

3 `/sc:analyze src/api --focus security`

auth and input-validation review

4 `/sc:test --type integration --coverage`

end-to-end API checks

5 `/sc:document src/api --type api --style detailed`

API reference

✓ **Result:** A secure, tested, documented API.**B3 A React component, generated and checked**

● Beginner

When: A button, form, card or similar piece of interface.1 `/sc:implement "pricing card with monthly/yearly toggle" --type component --framework react --magic --c7`

component from modern UI patterns

2 `/sc:test --type e2e --play`

a real browser clicks the toggle

3 `/sc:improve src/components/PricingCard --type style --focus accessibility --interactive`

accessibility polish

✓ **Result:** A ready-to-use component that works in a real browser and meets accessibility basics.

💡 Magic needs a 21st.dev API key. Without it, the command still works using Claude's own skills.

B4 A Vue component

● Beginner

When: Same as above, for a Vue project.1 `/sc:implement "dashboard widget showing weekly sales" --type component --framework vue --c7`

component

2 `/sc:test --type unit --coverage`

unit tests run

✓ **Result:** A Vue component built to official conventions.

B5 Design a database and add a migration

● Intermediate

When: New tables, or changes to existing ones.1 `/sc:design billing-db --type database --format spec`

tables, keys, relations

2 `/sc:implement "billing tables migration" --type service --safe --with-tests --validate`

migration with a risk check first

3 `/sc:test --type integration`

migration runs against test data

✓ **Result:** A reviewed schema and a migration that has been risk-checked and tested.

💡 Commit before running migrations. The rulebook asks for a commit before risky operations.

B6 A Python tool or service

● Intermediate

When: A command-line tool, script or small service in Python.1 `/sc:implement "CSV importer that validates and loads customer rows" --type service --with-tests`

code + tests

2 `/sc:test --coverage`

coverage report

3 `/sc:analyze src/importer --focus quality`

quality review

✓ **Result:** Idiomatic, tested Python with a quality report.💡 Mention `@agent-python-expert` in your request to force the Python specialist.**B7 Connect a third-party service (payments, email, maps)**

● Intermediate

When: You must integrate an outside API or SDK.1 `/sc:explain "Stripe webhooks" --level basic --c7`

how it works, from official docs

2 `/sc:implement "Stripe webhook handler with signature check" --type api --safe --with-tests --c7`

integration code

3 `/sc:analyze src/payments --focus security`

secrets and validation review

4 `/sc:test --type integration`

verify with test events

✓ **Result:** A secure integration built from current official documentation.

B8 A landing page from scratch

● Beginner

When: A simple marketing page.1 `/sc:brainstorm "landing page for a dog-walking service" --depth shallow`

sections and goals

2 `/sc:design landing --type component --format code`

page structure

3 `/sc:implement "landing page with hero, services, prices and contact form" --type feature --magic`

the page

4 `/sc:test --type e2e --frontend-verify`

browser + speed + layout checks

5 `/sc:improve src/landing --type performance --interactive`

speed polish

✓ **Result:** A tested, fast landing page.**B9 Add tests to code that has none**

● Intermediate

When: Existing code with little or no test coverage.1 `/sc:test --coverage`

baseline: what is covered today

2 `/sc:analyze src/payments --focus quality`

where the risk is

3 `@agent-quality-engineer "design a test strategy for src/payments"`

what to test and why

4 `/sc:implement "unit tests for src/payments per the strategy" --type feature --with-tests`

writes the tests

5 `/sc:test --coverage`

coverage after

✓ **Result:** A real coverage increase on the riskiest code.💡 `/sc:test` only runs tests. `/sc:implement` is the command that writes new ones.**B10 A large feature without blowing the budget**

● Expert

When: Big feature, big codebase, limited tokens.1 `/sc:index-repo mode=update`

compact repository map

2 `/sc:load`

loads the map instead of every file

- 3 `/sc:task execute "multi-currency support" --delegate --token-efficient --concurrency 5`
parallel sub-agents, compressed output
- 4 `/sc:reflect --type completion`
is it really finished?
- 5 `/sc:save --type all --checkpoint`
save progress

✓ **Result:** A large change delivered with far less context spent re-reading the codebase.

🔗 CHAPTER 14

Recipes: fixing problems

Bugs, broken builds, slow pages and mystery errors. 11 recipes.

F1 Quick bug fix

● Beginner

When: Something is broken and you can describe it.

- 1 `/sc:troubleshoot "total shows NaN when cart is empty" --type bug --trace`
diagnosis + ranked fixes (no edits)
- 2 `/sc:troubleshoot "total shows NaN when cart is empty" --type bug --fix`
applies the fix you approve
- 3 `/sc:test --type all`
nothing else broke?
- 4 `/sc:git commit --smart-commit`
commit

✓ **Result:** A diagnosed, fixed and verified bug.

💡 Paste the exact error message into the quotes. It helps more than anything else.

F2 Urgent production problem

● Intermediate

When: Users are affected right now.

- 1 `/sc:git checkout -b hotfix/checkout-500`
safe branch
- 2 `/sc:troubleshoot "checkout returns 500 since 14:00" --type deployment --trace --think-hard`
deep diagnosis
- 3 `/sc:troubleshoot "checkout returns 500 since 14:00" --fix --safe-mode`
most cautious fix
- 4 `/sc:test --type all`
verify

5 `/sc:git commit --smart-commit`

commit the hotfix

✓ **Result:** A careful, minimal fix on its own branch, ready to deploy.

💡 Never let anyone, human or AI, skip or disable a failing test to get a hotfix out. The rules forbid it.

F3 The build is broken

● Beginner

When: Your project will not compile or package.

1 `/sc:build --verbose`

full error output

2 `/sc:troubleshoot "build fails: <paste first error>" --type build --trace`

why

3 `/sc:troubleshoot "build fails: <paste first error>" --type build --fix`

fix

4 `/sc:build --clean`

fresh build to confirm

✓ **Result:** A clean build again, with the cause explained.

F4 Failing or flaky tests

● Intermediate

When: Tests fail, or pass only sometimes.

1 `/sc:test --type unit`

which tests fail

2 `/sc:troubleshoot "OrderTotal test fails about 1 run in 5" --type bug --think --introspect`

hypotheses with visible reasoning

3 `/sc:troubleshoot "OrderTotal test fails about 1 run in 5" --type bug --fix`

fix the root cause

4 `/sc:test --type unit`

re-run several times

✓ **Result:** Stable tests, with the real cause fixed rather than the test silenced.

F5 Deployment fails

● Intermediate

When: It works locally but fails when deployed.

1 `/sc:troubleshoot "deploy fails at migrate step" --type deployment --trace`

diagnosis

2 `@agent-devops-architect "review our deploy pipeline for this failure"`

infrastructure view

3 `/sc:build --type prod --verbose --validate`

reproduce the production build locally

✓ **Result:** A clear reason for the failure and a reproducible production build.

F6 A slow web page

● Intermediate

When: Pages take too long to load or feel janky.

1 `/sc:analyze src --focus performance --format report`

likely hot spots

2 `/sc:troubleshoot "home page takes 4s to show content" --type performance --chrome`

real measurements

3 `/sc:improve src/pages/home --type performance --interactive`

targeted fixes you approve

4 `/sc:test --type e2e --play`

still works

✓ **Result:** Measured before-and-after speed, not guesswork.

💡 The `--chrome` flag is listed in the framework's flag file but not in `/sc:help`. If it seems ignored, just say "use Chrome DevTools".

F7 A slow API or back end

● Intermediate

When: Server responses are slow.

1 `/sc:analyze src/api --focus performance --depth deep`

where time goes

2 `/sc:troubleshoot "GET /orders takes 3s" --type performance --seq`

systematic bottleneck hunt

3 `/sc:improve src/api/orders --type performance --loop --iterations 3`

three improve-and-measure rounds

4 `/sc:test --coverage`

nothing broke

✓ **Result:** A faster endpoint with each improvement verified.

F8 Memory keeps growing (a leak)

● Expert

When: The app slows down or crashes after running a while.

1 `/sc:troubleshoot "worker memory grows 50MB per hour" --type performance --trace -
-ultrathink`

deepest investigation

2 `@agent-root-cause-analyst "confirm the leak source with evidence"`

evidence chain

3 `/sc:troubleshoot "worker memory grows 50MB per hour" --fix --validate`

risk-checked fix

✓ **Result:** The leak source proven with evidence, then fixed.

F9 Console errors or a broken layout

● Beginner

When: Red errors in the browser console, or things in the wrong place.

1 `/sc:troubleshoot "TypeError: cannot read map of undefined on /profile" --type bug --frontend-verify`

browser, profiler and code tools together

2 `/sc:troubleshoot "TypeError: cannot read map of undefined on /profile" --type bug --fix`

fix

3 `/sc:test --type e2e`

page works again

✓ **Result:** A clean console and a correct layout.

F10 An upgrade broke things

● Intermediate

When: You updated a library and now errors appear.

1 `/sc:research "React 19 upgrade breaking changes" --depth quick`

what changed

2 `/sc:troubleshoot "errors after upgrading React" --type build --c7`

diagnosis using current docs

3 `/sc:improve src --type maintainability --morph --interactive`

bulk-update old patterns

4 `/sc:test --type all`

verify

✓ **Result:** Your code updated to the new version's patterns.

F11 "I have no idea what's wrong"

● Beginner

When: You cannot even name the problem.

1 `/sc:recommend "the app was fine yesterday and now nothing loads"`

suggested command sequence

2 `/sc:troubleshoot "app shows a blank screen since yesterday" --introspect`

diagnosis with reasoning shown

3

```
/sc:git log
```

what changed since yesterday



Result: A named problem and a plan to fix it.

CHAPTER 15

Recipes: improving existing code

Refactoring, cleanup, security, accessibility and speed. 9 recipes.

I1 Refactor safely

● Intermediate

When: Code works but is hard to change.

1

```
/sc:test --type all
```

baseline: all green first

2

```
/sc:analyze src/orders --focus architecture
```

what to simplify

3

```
/sc:improve src/orders --type maintainability --safe --interactive
```

small approved steps

4

```
/sc:test --type all
```

still green

5

```
/sc:git commit --smart-commit
```

commit



Result: Simpler code that behaves exactly as before.



Ask for `@agent-refactoring-expert` if you want metrics on complexity before and after.

I2 Clean up a legacy codebase

● Expert

When: Old code nobody wants to touch.

1

```
/sc:index-repo
```

map the territory

2

```
/sc:analyze . --focus architecture --depth deep --ultrathink
```

full structural review

3

```
/sc:cleanup src --type all --interactive
```

remove dead code with approval

4

```
/sc:improve src/legacy --type maintainability --loop --iterations 3
```

gradual improvement

5 `/sc:test --type all --coverage`

verify

✓ **Result:** Less, cleaner code, with every change reviewed and tested.

I3 Security audit

● Intermediate

When: Before launch, after an incident, or on a schedule.

1 `/sc:analyze . --focus security --depth deep --think-hard`

vulnerability report by severity

2 `@agent-security-engineer "threat-model the login and payment flows"`

attacker's-eye view

3 `/sc:improve src/auth --type quality --focus security --validate`

fixes, risk-checked

4 `/sc:test --type e2e --play`

real auth flows still work

✓ **Result:** A prioritised list of security issues, with the important ones fixed.

💡 The original guide used `--type security` on `/sc:improve`. That value is not in the command's own list; `--focus security` is the documented way.

I4 Accessibility review

● Intermediate

When: You want everyone, including screen-reader and keyboard users, to manage.

1 `/sc:analyze src/components --focus quality --depth deep "check against WCAG 2.1 AA"`

issues found

2 `/sc:improve src/components --type quality --focus accessibility --interactive`

fixes you approve

3 `/sc:test --type e2e --play`

browser-based checks

✓ **Result:** Components that meet common accessibility standards.

I5 Remove dead code and tidy imports

● Beginner

When: Unused files and imports have piled up.

1 `/sc:cleanup src --type imports --safe`

unused imports removed automatically

2 `/sc:cleanup src --type code --interactive`

dead code, with questions

3 `/sc:test --type all`

verify

✓ **Result:** A leaner project with nothing important removed.

I6 Apply one change across many files

● Intermediate

When: Replace a pattern everywhere, enforce a style rule.

1 `/sc:select-tool "replace console.log with logger.info across the project" --explain`

confirms Morphllm is the right tool

2 `/sc:improve src --type style --morph "replace console.log with logger.info"`

bulk edit

3 `/sc:test --type all`

verify

✓ **Result:** A consistent change applied everywhere in one pass.

I7 Rename something everywhere

● Intermediate

When: A function, class or variable needs a better name.

1 `/sc:select-tool "rename UserService to AccountService" --analyze`

confirms Serena

2 `/sc:improve src --type maintainability --serena --safe "rename UserService to AccountService"`

meaning-aware rename, references included

3 `/sc:build`

still compiles

4 `/sc:test --type all`

still passes

✓ **Result:** A rename that updates every real reference and nothing else.

I8 Polish a screen until it feels right

● Beginner

When: The layout works but looks unfinished.

1 `/sc:improve src/components/Navbar --type style --loop --iterations 3 --magic`

three polish rounds

2 `/sc:test --type e2e --play`

still works

✓ **Result:** A visibly more polished component.

I9 Tighten types and catch errors early

● Intermediate

When: A JavaScript/TypeScript project with loose or missing types.**1** `/sc:analyze src --focus quality`

weak spots

2 `/sc:improve src --type quality --safe "add strict TypeScript types"`

typed code

3 `/sc:build`

type check passes

4 `/sc:test --type all`

verify

✓ **Result:** Stricter types, with mistakes caught at build time.

📖 CHAPTER 16

Recipes: understanding and documenting

*Getting to know code and writing it down. 8 recipes.***L1 Get to know an unfamiliar codebase**

● Beginner

When: Your first days on someone else's project.**1** `/sc:index-repo`

repository map

2 `/sc:load`

load it

3 `/sc:analyze . --focus architecture --depth quick`

overview of the structure

4 `/sc:explain src/core --level basic`

plain-language tour

5 `/sc:save --type learnings`

remember what you learned

✓ **Result:** A mental map of the project, saved for next time.**L2 Understand one confusing file**

● Beginner

When: A single file makes no sense to you.**1** `/sc:explain src/auth/token.ts --level basic --format examples`

explanation with examples

2 `/sc:explain src/auth/token.ts --level intermediate`

deeper once the basics click

✓ **Result:** Understanding built up in stages.

L3 Learn a new framework or concept

● Beginner

When: You are new to a technology.

1 `/sc:explain "React Server Components" --level basic --c7`

explanation from current docs

2 `@agent-learning-guide "build me a 5-step learning path for React Server Components"`

structured path

3 `@agent-socratic-mentor "quiz me on what I just learned"`

check your understanding

✓ **Result:** Real understanding rather than copied snippets.

L4 Write a README

● Beginner

When: Your project needs a front page.

1 `/sc:index . --type readme --format md`

README draft

2 `/sc:document src --type guide --style brief`

short usage guide

✓ **Result:** A README and quick-start guide.

L5 Document an API

● Intermediate

When: Others need to call your endpoints.

1 `/sc:document src/api --type api --style detailed`

reference for each endpoint

2 `/sc:index . --type api --format md`

project-wide API index

✓ **Result:** Complete API reference documentation.

L6 Comment and document code after building

● Beginner

When: You just finished a feature.

1 `/sc:document src/auth/login.ts --type inline --style detailed`

inline comments

2 `/sc:document src/auth --type external --style brief`

short module guide

3 `/sc:index . --type docs --format md`

refresh project docs

✓ **Result:** Code that the next person can follow.

L7 Prepare for a code review

● Intermediate

When: Before you ask a colleague to review.

1 `/sc:git diff`

see what changed

2 `/sc:analyze src --focus quality --depth quick`

obvious issues

3 `@agent-self-review "check my changes against the original requirements"`

post-build checklist

4 `/sc:reflect --type completion`

is it truly done?

✓ **Result:** Cleaner changes and fewer review comments.

L8 Get coached instead of given answers

● Beginner

When: You want to become a better developer, not just finish the task.

1 `@agent-socratic-mentor "review my OrderService class"`

questions that lead you to the problems

2 `/sc:explain src/orders/OrderService.ts --level intermediate --format interactive`

interactive explanation

✓ **Result:** Principles you discovered yourself, which stick better.

🔗 CHAPTER 17

Recipes: shipping and working across days

Commits, releases, hand-offs and session routines. 6 recipes.

S1 Daily start and finish routine

● Beginner

When: Every working day.

1 `/sc:load`

morning: restore yesterday's context

...your work...

3 `/sc:save --checkpoint`

every 30 minutes or before a risky step

4 `/sc:reflect --type session`

end of day: what got done?

5 `/sc:save --type all --summarize`

end of day: save everything

✓ **Result:** Tomorrow starts where today ended, with nothing to re-explain.

S2 A feature that takes several days

● Intermediate

When: Work that spans more than one session.

1 `/sc:workflow "export to Excel" --strategy systematic`

day 1: plan

2 `/sc:save --type all --checkpoint`

day 1: save

3 `/sc:load --type checkpoint`

day 2: restore exactly where you stopped

4 `/sc:implement "export to Excel" --type feature --with-tests`

day 2: build

5 `/sc:reflect --type completion`

last day: confirm done

✓ **Result:** Continuity across days without losing context.

S3 Hand work over to a teammate

● Intermediate

When: Someone else will continue your work.

1 `/sc:reflect --type session`

what is done and what is not

2 `/sc:document src/reports --type guide --style detailed`

how the new code works

3 `/sc:save --type all --summarize`

summary for the next session

4 `/sc:git commit --smart-commit`

commit everything

✓ **Result:** A clean hand-over with documentation and a status summary.

S4 Commit and push

● Beginner

When: Your change is ready.

- 1 `/sc:git status`
what changed
- 2 `/sc:git add .`
stage changes
- 3 `/sc:git commit --smart-commit`
conventional commit message written for you
- 4 `/sc:git push`
send to the remote

 **Result:** A clean, well-described commit on the remote.**S5 Pre-release checklist**

● Intermediate

When: Before tagging a release.

- 1 `/sc:test --type all --coverage`
everything passes
- 2 `/sc:analyze . --focus security --depth deep`
no known vulnerabilities
- 3 `/sc:build --type prod --clean --optimize --validate`
production build, risk-checked
- 4 `/sc:document . --type external --style brief`
release notes draft
- 5 `/sc:git tag v1.4.0`
tag the release

 **Result:** A tested, audited, documented release.**S6 Production build with maximum care**

● Intermediate

When: Building for a critical environment.

- 1 `/sc:build --type prod --safe-mode --validate --focus security`
most cautious build
- 2 `/sc:test --type all`
verify

 **Result:** A production build made under the strictest settings.

CHAPTER 18

Recipes: research, business and specifications

Evidence and expert review before you commit. 5 recipes.

R1 A cited research report

● Beginner

When: You need facts, not opinions.

1 `/sc:research "how small charities accept online donations in 2026" --depth deep --strategy unified`

multi-round, cited report

✓ **Result:** A structured report in claudedocs/ with sources rated for credibility.

💡 Depth guide: quick (1 round, ~2 min), standard (2–3), deep (3–4), exhaustive (5 rounds, 50+ sources, ~10 min).

R2 Competitor analysis into strategy

● Intermediate

When: You need to understand the market before positioning.

1 `/sc:research "AI coding assistant competitors: pricing and features" --depth exhaustive`

full landscape

2 `/sc:business-panel @claudedocs/research_competitors.md --mode discussion --experts "porter,godin,christensen"`

strategic options

3 `/sc:business-panel @claudedocs/research_competitors.md --synthesis-only`

just the conclusions

✓ **Result:** A market picture and a short list of strategic options.

R3 Stress-test a business plan

● Intermediate

When: You want the weak spots found before an investor finds them.

1 `/sc:business-panel @business_plan.md --mode debate --experts "taleb,porter,collins"`

adversarial critique

2 `/sc:business-panel @business_plan.md --mode socratic`

questions you must be able to answer

3 `/sc:business-panel @business_plan.md --synthesis-only --structured`

prioritised recommendations

✓ **Result:** A list of risks and the questions you need answers to.

R4 Quality gate for a specification

● Intermediate

When: Before any code is written from a spec.

- 1 `/sc:spec-panel @auth_spec.yml --mode critique --focus requirements`
gaps and ambiguities
- 2 `/sc:spec-panel @auth_spec_v2.yml --mode discussion --iterations 2`
review of your revision
- 3 `/sc:design auth-api --type api --format spec`
design from the approved spec
- 4 `/sc:implement "authentication API" --type api --with-tests`
build

✓ **Result:** A spec that scored well before a single line was written.**R5** Write an architecture decision record

● Expert

When: A big technical decision others must understand later.

- 1 `/sc:research "event sourcing trade-offs for order systems" --depth deep`
evidence
- 2 `/sc:design order-events --type architecture --format spec --think-hard`
proposed design
- 3 `/sc:spec-panel @claudedocs/order_events_adr.md --focus architecture --experts "fowler,nygard,newman"`
expert critique

✓ **Result:** A documented, reviewed decision with the reasoning preserved.

CHAPTER 19

Recipes: special set-ups

*Small budgets, no extra tools, maximum caution, maximum depth. 4 recipes.***X1** Working on a small token budget

● Intermediate

When: You are on a limited plan or a very long session.

- 1 `/sc:index-repo mode=quick`
small repository map
- 2 `/sc:analyze src/cart --scope module --uc`
narrow scope, compressed output

3 `/sc:implement "fix rounding in cart totals" --no-mcp`

no extra tools loaded

✓ **Result:** The same work with noticeably less context used.

💡 Starting a fresh Claude Code session also resets the context window.

X2 No extra tools installed

● Beginner

When: You installed SuperClaude but none of the MCP servers.

1 `/sc:analyze src --no-mcp`

built-in tools only

2 `/sc:troubleshoot "login fails" --type bug --no-mcp`

diagnosis without extras

✓ **Result:** Everything still works. Some commands are simply less powerful.

💡 Session memory (`/sc:load` , `/sc:save`) needs Serena; everything else degrades gracefully.

X3 Maximum caution: change nothing without asking

● Beginner

When: You are new, nervous, or working on something important.

1 `/sc:troubleshoot "describe problem"`

diagnosis only, no edits

2 `/sc:improve src --type quality --safe --interactive`

every change needs approval

3 `/sc:build --safe-mode --validate`

most cautious execution

✓ **Result:** Full control: nothing changes until you say yes.

X4 Maximum depth for critical work

● Expert

When: A redesign or a bug that has defeated everyone.

1 `/sc:troubleshoot "intermittent data corruption in sync job" --type bug --trace --ultrathink --all-mcp --introspect`

deepest analysis, every tool, reasoning shown

2 `/sc:reflect --type task --validate`

check the conclusion

✓ **Result:** The most thorough analysis the framework can do. It costs the most tokens.

PART IV

Help and reference

Fixes, answers, definitions, pocket cards and how this guide was checked.



20

Troubleshooting

51



21

Frequently asked questions

52



22

Glossary

53



23

Pocket reference cards

55



24

How this guide was verified

57

🔧 CHAPTER 20

Troubleshooting

Symptoms, likely causes and the exact fix.

Find the symptom, read the likely cause, apply the fix. Most problems are solved by the first three rows plus a restart.

Symptom	Likely cause	What to do
/sc: commands don't appear	Claude Code hasn't reloaded, or files went to a different folder.	Quit Claude Code completely and reopen. Check with <code>superclaude install --list</code> and <code>ls ~/.claude/commands/sc/</code> . With a custom <code>CLAUDE_CONFIG_DIR</code> , install there: <code>superclaude install --target "\$CLAUDE_CONFIG_DIR/commands/sc"</code> .
superclaude: command not found	The install folder isn't on your PATH.	Run <code>pipx ensurepath</code> and open a new terminal, or use <code>python -m superclaude install</code> .
Installer refuses your Python	Python older than 3.10.	Install Python 3.10+ and use it explicitly, for example <code>py -3.12 -m pip install superclaude</code> on Windows.
An update changed nothing	<code>superclaude install</code> skips existing files.	Run <code>superclaude update</code> , then restart Claude Code.
Permission denied during install	Installing into a system folder.	Use pipx, or <code>pip install --user superclaude</code> .
Health check	You want to know what is wrong.	Run <code>superclaude doctor --verbose</code> for a detailed report.
"Claude CLI not found" when adding tools	The tool installer calls the <code>claude</code> command.	Make sure <code>claude --version</code> works in the same terminal, and that Node.js is 18 or newer.
"Failed to spawn: serena"	Serena was registered with the wrong runner.	<code>claude mcp remove serena</code> , then <code>claude mcp add serena -- uvx -- from git+https://github.com/oraios/serena serena start-mcp-server --context ide-assistant</code> . Requires uv.
Research or UI tools return nothing	Missing API key.	Set <code>TAVILY_API_KEY</code> , <code>TWENTY_FIRST_API_KEY</code> or <code>MORPH_API_KEY</code> , then reinstall that tool.
AIRIS gateway won't start	Docker isn't running.	Start Docker Desktop (or the Docker service), then <code>superclaude mcp --servers airis-mcp-gateway</code> .

Symptom	Likely cause	What to do
/sc:load restores nothing	Serena isn't installed, or nothing was saved yet.	Install Serena, then run <code>/sc:save</code> at least once.
/index-repo is unknown	The file documents itself without the prefix.	Type <code>/sc:index-repo</code> .
/sc:help lists fewer commands than this guide	Its table is hand-written and covers 24 of 30.	Use <code>superclaude install --list</code> for the true list.
A command stopped without writing code	Intended: it is a planning or look-only command.	See the hand-off table on page 17 for what to run next.
Replies became short and full of symbols	Token Efficiency mode turned on (context over 75% full).	Start a new session, or ask Claude to answer in full sentences.
A specialist refuses part of a task	Each specialist has deliberate limits.	Call the right one, for example <code>@agent-backend-architect</code> .
A flag seems to be ignored	Flags are read as instructions; three are missing from <code>/sc:help</code> .	Say what you want in words as well as with the flag.
install.sh fails on Windows	It is a bash script.	Use pip or pipx, or run it inside Git Bash.

? CHAPTER 21

Frequently asked questions

Short answers to what newcomers ask most.

? Is SuperClaude made by Anthropic?

No. It is an independent open-source project under the MIT licence. Anthropic makes Claude Code; it does not support SuperClaude.

? Does it cost anything?

SuperClaude itself is free. Some helper tools need keys from their providers (Tavily, 21st.dev, Morph), which may have free tiers or charges.

? Will it change my Claude plan, model or bill?

No. It is configuration only. Deeper flags such as `--ultrathink` do use more tokens on that task, because they do more work.

? Do I need to know how to code?

Not for research, business review, explanation or planning commands. For building, you will understand more if you can read code, but the beginner recipes explain each step.

? Do I need all ten helper tools?

No. Every command works without them. Start with Context7 and Sequential; add others when a recipe calls for them.

? Is it safe on an existing project?

Look-only and plan-only commands never touch files. The house rules ask for feature branches and commits before risky steps. Still review changes before committing, as with any tool that writes code.

? Why does a command stop instead of finishing the job?

Deliberately, so you can review a plan, diagnosis or spec before code changes. Chapter 5 lists every stopping point.

? What is the difference between `/sc:pm` and `/sc:task`?

`/sc:pm` is the coordinator for a whole goal across a session. `/sc:task` runs one defined job and stops.

? Are `/implement` and `/sc:implement` different?

No. The short form appears in some documentation; the installed name is `/sc:implement`.

? Can I use only part of it?

Yes. Use one command and ignore the rest. Nothing depends on using the whole framework.

? Where do reports and plans end up?

In a `claudedocs/` folder in your project, for example `claudedocs/research_*.md` and `claudedocs/workflow_*.md`.

? How do I know which version I have?

Run `superclaude` version in a terminal. This guide covers 4.3.0.

? What is coming next?

A TypeScript plugin edition (v5) is in development with no release date. Until then, install with `pip` or `pipx` as described here.

? Does it work with other AI tools?

This framework targets Claude Code. Sister projects exist for other assistants (SuperGemini, SuperQwen), maintained separately.

≡ CHAPTER 22

Glossary

Every technical term in this guide, in everyday words.

Agent	See Specialist. SuperClaude's word for an expert role.
API	A defined way for programs to talk to each other, for example a list of web addresses a server answers.
API key	A secret code a service gives you so it knows who is using it.
Architecture	The overall shape of a system: its main parts and how they connect.
Branch	A separate line of work in git, so changes don't affect the main version until you merge them.
Build	Turning source code into something that runs or can be shipped.

CI/CD	Continuous integration and delivery: automatic testing and deployment whenever code changes.
Claude Code	Anthropic's tool that lets Claude work directly on files in a project folder.
Commit	A saved snapshot of changes in git, with a message describing them.
Context window	How much text Claude can keep in mind at once in a conversation.
Core Web Vitals	Google's measures of page experience: loading (LCP), stability (CLS) and responsiveness (INP).
Coverage	The share of your code that tests actually exercise.
Dead code	Code that is never used and can be removed.
E2E test	End-to-end test: drives the real application the way a user would.
Flag	A switch starting with -- that changes how a command behaves.
Git	The most common tool for tracking changes to code over time.
Hop	One round of follow-up research, using what was found to search again.
MCP	Model Context Protocol, the standard for plugging extra tools into Claude Code.
Migration	A scripted change to a database's structure.
Mode	A working style, such as questioning or compressed, that changes how Claude behaves.
MVP	Minimum viable product: the smallest version that is still useful.
OWASP Top 10	The best-known list of the most serious web security risks.
PATH	The list of folders your terminal searches for programs.
pipx	A tool that installs Python command-line programs in their own tidy space.
Refactor	Restructuring code to make it simpler without changing what it does.
Repository (repo)	A project folder tracked by git.
Slash command	A shortcut typed in Claude Code that starts with /.
Specialist	A set of instructions that makes Claude act as a particular expert.
Specification (spec)	A written description of exactly what should be built.
Terminal	The text window where you type commands to your computer.
Token	A small chunk of text, roughly three-quarters of a word. Usage and limits are counted in tokens.
WCAG 2.1 AA	The accessibility standard most often required by law.

CHAPTER 23

Pocket reference cards

Dense one-page summaries to print and keep beside you.
Print these pages and keep them beside your keyboard.

☆ The eight you'll use most (Claude Code)

- `/sc:load` start a session
- `/sc:recommend ""` not sure what to do
- `/sc:brainstorm ""` shape an idea
- `/sc:implement ""` build it
- `/sc:troubleshoot ""` find out why
- `/sc:test` run tests
- `/sc:git commit --smart-commit` commit
- `/sc:save --checkpoint` save progress

⚙️ Flags worth remembering

- `--think / --think-hard / --ultrathink` think more
- `--c7` official docs
- `--seq` step-by-step reasoning
- `--magic` UI generation
- `--play` browser tests
- `--serena` meaning-aware edits, memory
- `--morph` bulk edits
- `--safe-mode` most cautious
- `--validate` risk check first
- `--loop --iterations 3` repeat and refine
- `--uc` short answers
- `--no-mcp` no extra tools
- `--focus security` one concern
- `--scope module` narrow view

📄 Terminal commands

- `pipx install superclaude` install
- `superclaude install` copy commands
- `superclaude update` after upgrading
- `superclaude install --list` see what's installed
- `superclaude doctor` health check
- `superclaude mcp --list` helper tools
- `superclaude mcp --servers NAME` add a tool
- `superclaude version` version

📖 All 30 commands

Command	In one line	Effect
<code>/sc</code>	A short menu of the framework.	Look only
<code>/sc:agent</code>	A session controller: checks git, investigates, plans, builds and reviews a task.	Changes files
<code>/sc:analyze</code>	Reads your code and reports problems by severity: quality, security, performance or architecture.	Look only
<code>/sc:brainstorm</code>	Asks you questions, one round at a time, until a vague idea becomes a clear list of requirements.	Plans only
<code>/sc:build</code>	Compiles and packages your project using the build setup it already has.	Changes files

Command	In one line	Effect
<code>/sc:business-panel</code>	Runs a document past nine simulated business thinkers (Porter, Christensen, Drucker, Godin, Kim & Mauborgne, Collins, Taleb, Meadows, Doumont).	 Plans only
<code>/sc:cleanup</code>	Removes dead code and unused imports, and tidies file structure.	 Changes files
<code>/sc:design</code>	Designs the structure of something: the architecture, an API, a component or a database.	 Plans only
<code>/sc:document</code>	Writes documentation for one specific thing: code comments, an API reference or a user guide.	 Changes files
<code>/sc:estimate</code>	Estimates time, effort or complexity, with a breakdown and a confidence level.	 Look only
<code>/sc:explain</code>	Explains code or a concept at the level you choose, from beginner to advanced.	 Look only
<code>/sc:git</code>	Runs git for you and writes tidy, conventional commit messages.	 Changes files
<code>/sc:help</code>	Prints a table of commands and flags.	 Look only
<code>/sc:implement</code>	Writes the code for a feature.	 Changes files
<code>/sc:improve</code>	Makes existing code better: cleaner, faster or easier to maintain.	 Changes files
<code>/sc:index</code>	Generates documentation for the whole project: a docs set, an API index, a structure map or a README.	 Changes files
<code>/sc:index-repo</code>	Writes a short map of your repository (PROJECT_INDEX.md and .json) for cheaper sessions.	 Changes files
<code>/sc:load</code>	Opens your project and restores what was saved last time.	 Memory
<code>/sc:pm</code>	The project manager: breaks a plain-English goal into steps and calls the right specialists.	 Changes files
<code>/sc:recommend</code>	Describe your problem in everyday words and it suggests which commands to run, in what order.	 Look only
<code>/sc:reflect</code>	A sanity check: is the current approach still on target, and is the task really finished?	 Look only
<code>/sc:research</code>	Searches the web and writes a cited research report.	 Plans only
<code>/sc:save</code>	Saves what was learned and done, so the next <code>/sc:load</code> can pick up seamlessly.	 Memory
<code>/sc:select-tool</code>	Decides which editing tool suits a change: Serena for meaning-aware edits (like renames), Morphllm for fast find-and-replace across files.	 Look only
<code>/sc:spawn</code>	Splits a very large job into epics, stories and tasks, and decides what can run in parallel.	 Plans only
<code>/sc:spec-panel</code>	A panel of simulated software-engineering authors scores a specification for clarity, completeness, testability and consistency (0–10 each).	 Plans only

Command	In one line	Effect
<code>/sc:task</code>	Runs one clearly defined job from start to finish, coordinating specialists, then stops.	 Changes files
<code>/sc:test</code>	Finds your test runner, runs your existing tests and reports coverage.	 Look only
<code>/sc:troubleshoot</code>	Diagnoses why something is broken and lists ranked fixes.	 Look only
<code>/sc:workflow</code>	Turns a feature description or requirements document into a step-by-step build plan with dependencies.	 Plans only

🔗 CHAPTER 24

How this guide was verified

Sources, corrections to the previous edition, and known gaps.

Every command name, syntax line, flag, installer option and version number in this edition was checked against the SuperClaude source code rather than against secondary documentation.

🔍 What was checked, and how

- The project repository `SuperClaude-0rg/SuperClaude_Framework`, master branch, commit `fe68862` (27 September 2026): all 30 command files, 20 specialist files, 10 tool files, the flag, rule and mode files, and the installer's Python source.
- The published package on PyPI: latest version 4.3.0, requires Python 3.10+, MIT licence.
- The changelog entries for 4.2.0 and 4.3.0, and the project's Windows and Serena troubleshooting pages.
- Every recipe uses only commands and flags present in that source. Where the source contradicts itself, both sides are shown.

✎ Corrections to revision 3.0

Topic	Revision 3.0 said	Verified fact
Python version	Required 3.8 or newer	Requires 3.10 or newer (package metadata and installer)
Updating	<code>pip install --upgrade</code> , then <code>superclaude install</code>	install skips existing files; use <code>superclaude update</code> (or <code>install --force</code>)
<code>/sc:help</code>	Described as authoritative for your install	A hand-written table of 24 commands; six are missing
Auto-start	<code>/sc:pm</code> and <code>/sc:agent</code> run automatically each session	Only in the planned plugin edition; with <code>pip/pipx</code> you type them
deep-research	Near-identical duplicate	A 31-line summary of the 184-line <code>deep-research-agent</code>
Introspection markers	Shown as <code>[thinking]</code> , <code>[target]</code> and similar	The source uses small symbol markers; described here in words
<code>/sc:improve</code> flags	Workflows used <code>--type security</code> and <code>--preview</code>	Neither is in the command's syntax; this guide uses <code>--focus security</code> and <code>--interactive</code>

Topic	Revision 3.0 said	Verified fact
Helper-tool setup	Not covered	superclaude mcp options, API keys, Node 18+, uv and Docker now documented
Airis Agent and MindBase	Listed like the other tools	Available only through the AIRIS gateway (Docker)
Modes	Mentioned only as flags	Seven modes now explained as a concept (chapter 7)
CLI commands	Only install was shown	doctor, version, update, install --list, install-skill and mcp documented
Version and licence	No version stated	v4.3.0, released 22 March 2026, MIT licence
Installed files	Implied rules and modes are installed	The 4.3.0 installer copies commands and specialists only; flagged for local checking
Recipes	12 workflows	59 recipes in eight groups, plus a situation finder

 Known gaps and claims not independently tested

- Token savings (for example “58K to 3K” for repository indexing, “30–50%” for compressed mode) are the project's own figures.
- Automatic triggers (such as modes switching on above 75% context) describe intended behaviour; Claude may not always follow them exactly.
- SuperClaude changes often. Run `superclaude install --list` and `superclaude version` to confirm what you have.

 How to cite this guide

APA (7th edition)

Made in Jurgistan. (2026). The SuperClaude Field Guide (Version 4.0) [Computer software].
<https://Made-in-Jurgistan.github.io/superclaude-field-guide/>

TEXT

BibTeX

```
@software{Made_in_Jurgistan_The_SuperClaude_2026,  
  author = {Made in Jurgistan},  
  month = {10},  
  title = {{The SuperClaude Field Guide}},  
  url = {https://Made-in-Jurgistan.github.io/superclaude-field-guide/},  
  version = {4.0},  
  year = {2026}  
}
```

BIBTEX

The SuperClaude Field Guide

Edition 4.0, 3 October 2026. Covers SuperClaude v4.3.0.
24 chapters, 59 recipes.

Made in Jurgistan. Licensed [CC BY-SA 4.0](#). SuperClaude is an independent MIT-licensed project, not affiliated with Anthropic. Accessible design: semantic headings, skip link, keyboard focus, text labels on every colour cue, tagged PDF with bookmarks.